

CPU Load and System Diagnostics Design Document

Document Version: 1.1

Last Updated: 2026-03-12

Related Code Files:

- `src/Ecum_Init_Main/EcuM_main_init.c` — Core diagnostics implementation (DWT time base, diagnostic update algorithm)
- `src/Ecum_Init_Main/EcuM_main_init.h` — Diagnostic data structures and public interface definitions
- `src/Ecum_Init_Main/EcuM_CpuLoadTest.c` — CPU load stress test task implementation
- `src/Ecum_Init_Main/EcuM_CpuLoadTest.h` — Stress test module configuration and interfaces
- `FlexCAN_Ip/FlexCAN_Ip_main.c` — Call entry point for diagnostic updates (10 ms periodic task)
- `PICC/Picc_Deamon/picc_main.c` — FreeRTOS task creation and scheduler startup
- `generate/include/FreeRTOSConfig.h` — FreeRTOS runtime statistics configuration (generated code, not editable)

1. Overview

Based on the current FreeRTOS project code, this document explains in detail the design principles and implementation mechanism of **Cortex-M7**-based CPU load monitoring and stress testing. This design is intended to provide the **S32G399A M7** core with high-precision task-level load statistics, overall system load evaluation, and dynamic load injection capability used to verify the currently available compute capacity.

2. Core Mechanism

2.1 Hardware-Level High-Precision Clock Source: DWT Cycle Counter

Instead of using the traditional operating system tick-based statistics method, which has a 1 ms tick granularity and introduces jitter and uneven measurement, the design uses the **DWT (Data Watchpoint and Trace) cycle counter built into the Cortex-M7 core** as the runtime statistics time base.

Property	Value	Code Reference
Register address	DWT_CYCCNT_REG → 0xE0001004	EcuM_main_init.c:91
Control register	DWT_CTRL_REG → 0xE0001000	EcuM_main_init.c:88
Debug enable	DEMCR_REG → 0xE000EDFC (bit 24 = TRCENA)	EcuM_main_init.c:93
Counting frequency	400 MHz (consistent with configCPU_CLOCK_HZ)	FreeRTOSConfig.h:45
Time resolution	2.5 ns	—
Overflow period	~10.73 seconds (2^32 / 400,000,000)	—

OS-level integration (connected through macros in `FreeRTOSConfig.h`):

```
portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() → vMainConfigureTimerForRunTimeStats()
portGET_RUN_TIME_COUNTER_VALUE()          → ulMainGetRunTimeCounterValue()
```

- `vMainConfigureTimerForRunTimeStats()` (`EcuM_main_init.c:106`): enables the DWT cycle counter.
 - i. Enable the Trace system: `DEMCR_REG |= TRCENA`
 - ii. Clear the counter: `DWT_CYCNT_REG = 0`
 - iii. Start counting: `DWT_CTRL_REG |= CYCNTENA`
- `ulMainGetRunTimeCounterValue()` (`EcuM_main_init.c:125`): directly returns the raw value of `DWT_CYCNT_REG` .

Key design constraint: `ulMainGetRunTimeCounterValue()` must **never perform any division**. FreeRTOS internally computes deltas through unsigned subtraction (`new - old`), and that operation is correct **only when the counter naturally wraps at 2^{32}** . Any division would introduce an artificial overflow point and break delta calculation correctness.

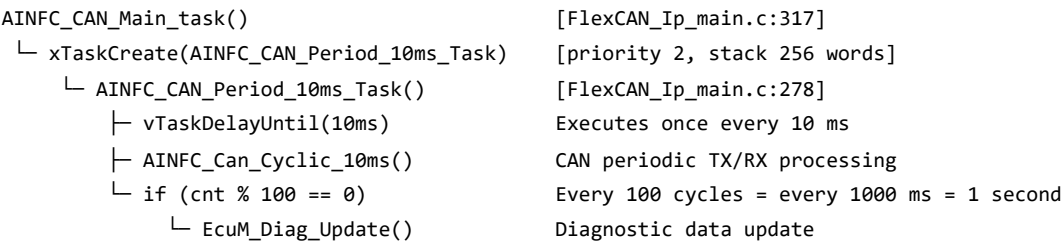
2.2 FreeRTOS Runtime Statistics Configuration Dependencies

The following `FreeRTOSConfig.h` settings are mandatory **prerequisites** for this scheme. They are located under `generate/include/` and cannot be modified directly because they are generated code.

Configuration Macro	Value	Purpose
<code>configGENERATE_RUN_TIME_STATS</code>	1	Enable runtime statistics
<code>configUSE_TRACE_FACILITY</code>	1	Enable the <code>uxTaskGetSystemState()</code> API
<code>configUSE_STATS_FORMATTING_FUNCTIONS</code>	1	Enable statistics formatting functions
<code>INCLUDE_xTaskGetIdleTaskHandle</code>	1	Get the IDLE task handle (used to compute total load)
<code>configCPU_CLOCK_HZ</code>	400000000UL	CPU main frequency 400 MHz
<code>configTICK_RATE_HZ</code>	1000	Tick frequency 1 kHz (1 ms)
<code>configTOTAL_HEAP_SIZE</code>	16384 (16 KB)	Total heap size
<code>configMAX_TASK_NAME_LEN</code>	10	Maximum task name length

3. Diagnostic Update Mechanism

3.1 Call Chain



- **Base period:** $10\text{ ms task period} \times \text{ECUM_DIAG_UPDATE_PERIOD_MS} / 10 = 1000 / 10 = 100$ iterations → **the actual diagnostic update frequency is 1 second.**

- **Configuration macro:** `ECUM_DIAG_UPDATE_PERIOD_MS` (defined in `EcuM_main_init.h:40`), currently set to `1000U`, and can be adjusted to any integer multiple of 10.

3.2 In-Depth Analysis of the CPU Load Algorithm

This section derives the CPU load calculation process used by `EcuM_Diag_Update()` from the mathematical principle level.

3.2.1 Basic Concept: FreeRTOS Runtime Counter Model

After `configGENERATE_RUN_TIME_STATS = 1` is enabled in FreeRTOS, the kernel automatically performs the following operations **at every task context switch**:

```

| FreeRTOS scheduler (internal execution on each context |
| switch) |
| |
| Current time T_now = portGET_RUN_TIME_COUNTER_VALUE() |
|                   = ulMainGetRunTimeCounterValue() |
|                   = DWT_CYCCNT_REG (raw 400 MHz cycle |
|                   count) |
| |
| The accumulated value of the task being switched out |
| += (T_now - T_lastSwitchIn) |
| |
| This accumulated value is what |
| uxTaskGetSystemState() returns as |
| s_taskStatusArray[i].ulRunTimeCounter |
| |
| At the same time, the sum of the accumulated values of |
| all tasks = ulTotalRunTime |

```

Key understanding:

- `ulRunTimeCounter` is the accumulated DWT cycle count representing the CPU time actually consumed by each task **since its creation** (`uint32` type).
- `ulTotalRunTime` is the sum of the `ulRunTimeCounter` values of all tasks, including IDLE.
- These values are all `uint32`, and they overflow about every 10.73 seconds at 400 MHz.

3.2.2 Differential Solution Model: Why the Accumulated Value Cannot Be Used Directly

Root cause of why the accumulated value cannot be used directly:

```
CPU_Load_A = Task_A.ulRunTimeCounter / ulTotalRunTime × 100% ← Wrong!
```

Because both `ulRunTimeCounter` and `ulTotalRunTime` are `uint32`, they overflow about every 10.73 seconds. Once the system has been running longer than 10.73 seconds, these values wrap back to zero and restart counting. The resulting value is only a partial accumulation from the last overflow until now, not the total since system startup. Dividing by this wrapped and discontinuous value produces a percentage with no physical meaning.

Correct method: differential calculation:

This algorithm does not care about the total amount from beginning to end. It only cares about the **increment (delta) between two samples**:

```
Snapshot saved at call N:      prev_task   = ulRunTimeCounter_N
                               prev_total   = ulTotalRunTime_N
```

```
Obtained at call N+1:        curr_task    = ulRunTimeCounter_{N+1}
                               curr_total   = ulTotalRunTime_{N+1}
```

Delta calculation:

```
deltaTask  = curr_task  - prev_task    ← DWT cycles consumed by the task in this interval
deltaTotal = curr_total - prev_total    ← DWT cycles consumed by the whole system in this interval
```

CPU Load % = $\text{deltaTask} / \text{deltaTotal} \times 100\%$

Why uint32 overflow does not affect the differential:

Unsigned subtraction in C naturally has modulo arithmetic behavior. Assume the uint32 overflow boundary is $2^{32} = 4,294,967,296$:

Example: prev = 4,294,000,000, curr = 500,000 (already overflowed)

Mathematically, the true delta = $4,294,967,296 - 4,294,000,000 + 500,000 = 1,467,296$

uint32 subtraction in C:

```
curr - prev = 500,000 - 4,294,000,000
             = 500,000 + (2^32 - 4,294,000,000)  [two's complement arithmetic]
             = 500,000 + 967,296
             = 1,467,296 ✓ Exactly matches the true value
```

Only prerequisite: the actual time elapsed between two samples must be < 10.73 seconds, which is one overflow period. The sampling interval in this system is 1 second, so the requirement is comfortably satisfied.

3.2.3 Anti-Overflow Design for Percentage Division

The most intuitive percentage formula is:

```
cpuLoadPercent = (deltaTask × 100) / deltaTotal
```

But this has a **32-bit integer overflow risk**:

At a 1-second sampling interval, deltaTotal is about 400,000,000 (400 MHz × 1 s)
deltaTask can also approach 400,000,000

```
deltaTask × 100 = 400,000,000 × 100 = 40,000,000,000
                ^^^^^^^^^^^^^^^^^
                Exceeds the uint32 maximum 4,294,967,295 → overflow!
```

Actual implementation in the code: divide first, then multiply in reverse form (EcuM_main_init.c:272):

```
divisor = deltaTotalRuntime / 100U;    // First shrink the denominator by 100x
// Example: divisor = 400,000,000 / 100 = 4,000,000

cpuLoadPercent = deltaTaskRuntime / divisor;
// Example: 40,000,000 / 4,000,000 = 10 → i.e. 10%
```

This keeps both the dividend and divisor of `deltaTaskRuntime / divisor` within the `uint32` range, so no overflow occurs.

Equivalent mathematical derivation:

```
deltaTask / divisor = deltaTask / (deltaTotal / 100)
                    = deltaTask × 100 / deltaTotal
                    = CPU Load %
```

3.2.4 Rounding Formula Derivation

Actual formula in the code (`EcuM_main_init.c:334-337`):

```
cpuLoadPercent = (uint8)(((unsigned long long)deltaTaskRuntime * 10ULL) / divisor + 5ULL) / 10ULL);
```

Step-by-step breakdown:

```
Step 1: temp1 = deltaTaskRuntime × 10  (increase precision by one digit for rounding)
Step 2: temp2 = temp1 / divisor        (get a value 10x the final result, keeping 1 decimal place)
Step 3: temp3 = temp2 + 5              (add 5, equivalent to +0.5 for rounding)
Step 4: result = temp3 / 10            (divide by 10 to remove the factor introduced in Step 1)
```

Numeric example: assume the system has 3 tasks during a 1-second sampling interval:

```
deltaTotalRuntime = 400,000,000  (1 second × 400 MHz)
divisor           = 400,000,000 / 100 = 4,000,000
```

Task A (`deltaTask = 120,400,000` → actual ratio 30.1%):

```
temp1 = 120,400,000 × 10 = 1,204,000,000  (note: unsigned long long is used, so no overflow)
temp2 = 1,204,000,000 / 4,000,000 = 301
temp3 = 301 + 5 = 306
result = 306 / 10 = 30  → output 30% (30.1% rounded to 30%) ✓
```

Task B (`deltaTask = 180,600,000` → actual ratio 45.15%):

```
temp1 = 180,600,000 × 10 = 1,806,000,000
temp2 = 1,806,000,000 / 4,000,000 = 451
temp3 = 451 + 5 = 456
result = 456 / 10 = 45  → output 45% ✓
```

IDLE task (`deltaTask = 99,000,000` → actual ratio 24.75%):

```
temp1 = 99,000,000 × 10 = 990,000,000
temp2 = 990,000,000 / 4,000,000 = 247
temp3 = 247 + 5 = 252
result = 252 / 10 = 25  → output 25% (24.75% rounded to 25%) ✓
```

Why Step 1 uses `unsigned long long` :

`deltaTaskRuntime` is explicitly promoted to 64-bit before multiplying by 10, ensuring no overflow. Since `deltaTaskRuntime` can be as large as about 400,000,000, multiplying by 10 yields 4,000,000,000, which is already near the edge of the `uint32` range (4,294,967,295). Using 64-bit arithmetic is completely safe.

Result clamp (`EcuM_main_init.c:339-341`): if the calculated result is > 100 , it is forcibly limited to 100 to handle extreme boundary-case division artifacts.

3.2.5 Overall CPU Load: IDLE Reverse-Deduction Method

Source of `idleCpuPercent` :

`idleCpuPercent` is not calculated separately. It is extracted "along the way" from inside the **per-task traversal loop** described in Section 3.2.4. The full data flow is as follows:

At the entry of `EcuM_Diag_Update()`:

```
uint8 idleCpuPercent = 0U;                ← initialized to 0 (line 208)
idleHandle = xTaskGetIdleTaskHandle();    ← get the IDLE task handle (line 279)

for (i = 0; i < uxArraySize; i++) {      ← iterate over all tasks
    currentHandle = s_taskStatusArray[i].xHandle;

    // ... calculate cpuLoadPercent for task[i]
    //      using the formula from Section 3.2.4 ...

    // ★ Key matching point (line 362-364):
    if (currentHandle == idleHandle) {    ← direct handle comparison
        idleCpuPercent = g_ecuMDiag.tasks[i].cpuLoadPercent;
        //      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        //      Directly use the just-calculated percentage, with no extra calculation
    }
}

// After the loop ends:
overallCpuLoad = 100 - idleCpuPercent;    ← (line 375-376)
```

Core points:

- In FreeRTOS, the IDLE task participates in `uxTaskGetSystemState()` statistics exactly like a normal task. It has its own `ulRunTimeCounter`, which records the total time that the IDLE task actually got CPU execution.
- During the traversal loop, the `cpuLoadPercent` of the IDLE task is computed using **exactly the same differential formula as all other tasks** (that is, the rounding formula from Section 3.2.4).
- After the calculation, the code identifies the IDLE task through **exact handle matching** with `currentHandle == idleHandle`, and captures its `cpuLoadPercent` into the local variable `idleCpuPercent`.
- The identification method is **handle comparison** (the pointer returned by `xTaskGetIdleTaskHandle()`), **not dependent on task name or priority**. Even if there are other user-created tasks with priority 0, there is no ambiguity.

Design choice: why use `100 - idle%` instead of summing all tasks:

The percentage of each task has already gone through integer division truncation, or truncation after rounding. If there are N tasks, the per-task errors accumulate when summed. For example, if 3 tasks are each 33.3%, truncating each one gives 33%, and the sum becomes 99% instead of 100%.

Reverse-deduction method (EcuM_main_init.c:375-376):

```
overallCpuLoad = 100 - idleCpuPercent
```

- 100 - idle ratio directly gives the load ratio, with only a single integer truncation error from the IDLE task itself, so the overall precision is better.

Numeric example (reusing the example from Section 3.2.4):

System with 3 tasks + IDLE during a 1-second sampling interval:

```
Task A cpuLoadPercent = 30%
```

```
Task B cpuLoadPercent = 45%
```

```
IDLE   cpuLoadPercent = 25% ← when iterating to IDLE, handle match succeeds,  
                               idleCpuPercent = 25
```

```
overallCpuLoad = 100 - 25 = 75%
```

Comparison with direct summation: 30 + 45 = 75% ← this example happens to match

But with more tasks, truncation error makes direct summation drift away

3.2.6 Dynamic Task Tracking Mechanism

Problem background: after `PICC_Init_Task` finishes initialization, it calls `vTaskDelete(NULL)` to delete itself. If other tasks are created later, FreeRTOS may reuse the reclaimed TCB memory for new tasks. If a traditional fixed index such as `xTaskNumber` is used for tracking, an identity mismatch can occur, and the historical runtime of the old task may be mistakenly attributed to the new task.

Solution used by this algorithm: `s_taskTracker[16]` handle-matching method:

```
s_taskTracker[j].handle      = TaskHandle_t    ← Unique identification key  
s_taskTracker[j].prevRuntime = uint32          ← The task's ulRunTimeCounter at the last sample  
s_taskTracker[j].accumCycles = uint64          ← 64-bit accumulated raw cycle count
```

Lookup flow (EcuM_main_init.c:305-313):

For the current `s_taskStatusArray[i]` being traversed:

Traverse `s_taskTracker[0..15]`:

```
if (tracker[j].handle == currentHandle) → existing record found, trackerSlot = j, break
```

```
if (tracker[j].handle == NULL && no empty slot has been found yet) → record the first empty slot
```

Result:

```
└ Existing record found → directly calculate the delta (deltaTask = curr - prev)
```

```
└ Not found → use the empty slot and initialize:
```

```
    handle = currentHandle
```

```
    prevRuntime = current ulRunTimeCounter ← makes the first delta = 0, avoiding a first-frame spike
```

```
    accumCycles = current ulRunTimeCounter ← preserves the already accumulated runtime
```

Why making the first delta = 0 is critical: when a new task is created, `ulRunTimeCounter` may already be non-zero by the time the first diagnostic sample is taken, because the task may have already accumulated some runtime. If `prevRuntime` were initialized to 0 instead of the current value, the first delta would be abnormally large and would create a one-shot output spike that interferes with observation.

3.2.7 Lossless 64-bit Cycle Accumulation: Why It Must Not Be Converted to Microseconds Too Early

Precision drift analysis:

If it is converted to microseconds at every delta step:

```
Wrong approach: accumMicroseconds += deltaTaskRuntime / 400;
```

For `deltaTaskRuntime = 1,599` (DWT cycles):

```
1,599 / 400 = 3 (integer division truncation, losing 399 cycles = 0.9975 µs)
```

Assume this truncation happens `400,000` times per second:

```
Accumulated error = 400,000 × 399 / 400 = 399,000 µs ≈ 0.4 seconds/second of drift!
```

Correct approach (used by this algorithm):

```
Correct: accumCycles += deltaTaskRuntime;           ← raw DWT cycles, no truncation
```

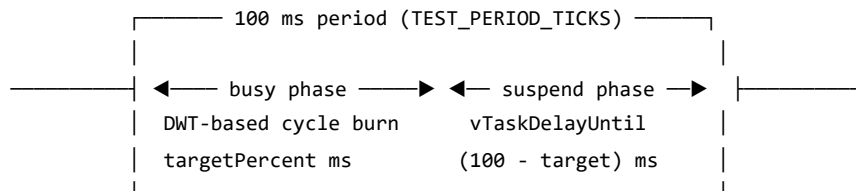
```
Final output: runtimeCounter = accumCycles / 400;   ← convert only once at the final output stage
```

The accumulated `accumCycles` value is completely lossless (unsigned long long = 64-bit, enough to hold about 1462 years of cycles).

The final `/ 400` causes only one truncation, with a maximum error of 399 cycles = 0.9975 µs, which is constant and does not drift.

3.2.8 Load Injection Timing Model of the Stress Test Task

`CpuLoadTest_Task` uses the following timing model to accurately generate a target percentage of CPU load:



Example: `g_cpuLoadTestPercent = 30` → busy for 30 ms + suspend for 70 ms = 100 ms

Busy phase precise control principle:

```
busyCycles = targetPercent × CYCLES_PER_MS
            = 30 × 400,000
            = 12,000,000 DWT cycles
```

```
startCycle = DWT_CYCCNT_REG;
```

```
while ((DWT_CYCCNT_REG - startCycle) < 12,000,000) {
```

```
    NOP; // Busy-spin to consume CPU; this section cannot be preempted unless interrupted by a higher-priority task
```

```
}
```

```
// Exit the loop after precisely 30 ms has elapsed
```

Suspend phase:

```
vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(100));
```

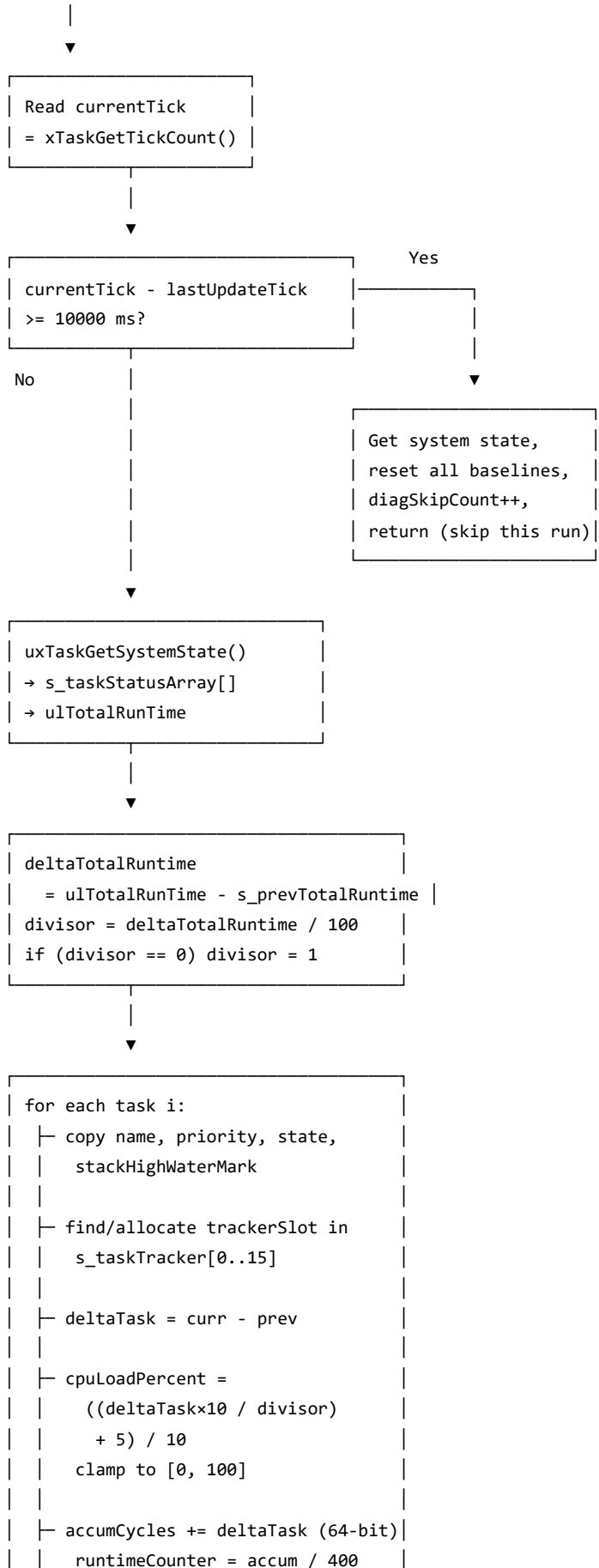
```
// Suspend this task until the next 100 ms boundary, giving CPU time back to other tasks and IDLE
```

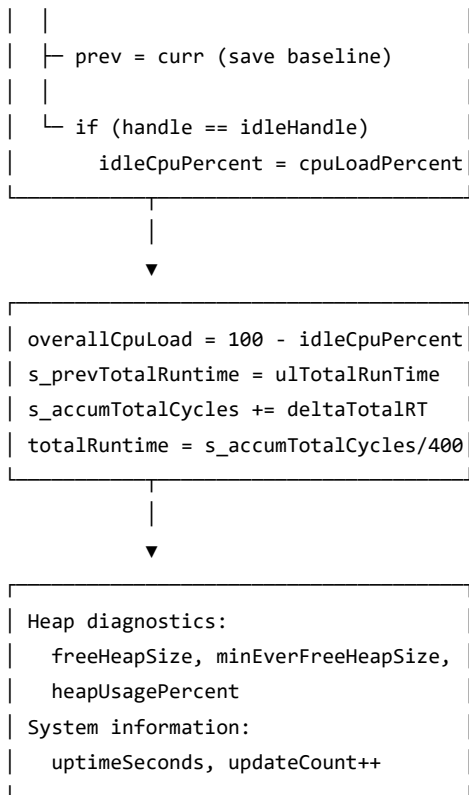
Precision analysis:

- The busy duration is controlled by hardware-level DWT timing, with 2.5 ns precision and no OS tick jitter influence.
- If the task is preempted by a higher-priority task during the busy phase, the preempted time is not excluded from `DWT_CYCCNT - startCycle` because DWT continues counting. As a result, the real wall-clock time before loop exit becomes greater than `targetPercent` ms. However, because this task has priority 1 and is only above IDLE, it is almost never preempted by user tasks, since all application tasks have priority ≥ 2 . Therefore, the busy-time accuracy remains high.

3.2.9 Complete Algorithm Flowchart

Entry of EcuM_Diag_Update()





3.3 Internal Data Structure Relationships

Array Name	Size	Purpose
s_taskStatusArray[]	ECUM_DIAG_MAX_TASKS = 8	Temporary buffer for FreeRTOS API return values
s_taskTracker[]	ECUM_MAX_TRACKED_TASKS = 16	Task handle → runtime history mapping, supporting dynamic task creation/deletion
g_ecuMDiag.tasks[]	ECUM_DIAG_MAX_TASKS = 8	Final diagnostic structure exposed externally

Design note: the capacity of `s_taskTracker` is 16, which is larger than the diagnostic output array capacity of 8. This is to handle task deletion and later recreation with changed handles. Even if the old and new handles refer to different tasks, the historical records in the tracker do not get confused.

4. Diagnostic Data Structures and Trace32 Debug View

4.1 Global Diagnostic Structure

All diagnostic results are collected into the global structure `g_ecuMDiag` (type `EcuM_DiagInfo_t`, defined in `EcuM_main_init.h:65`).

Memory placement: `__attribute__((section(".mcal_bss_no_cacheable")))` is used to force placement into a **non-cacheable** section.

- **Purpose:** after D-Cache is enabled on Cortex-M7, Trace32 reads SRAM directly. This prevents Trace32 from seeing stale data that still lives only in CPU cache and ensures reliable online debug visibility.

4.2 Complete Output Field List

Per-task diagnostic fields (EcuM_TaskDiag_t), each task contains:

Field Name	Type	Description
name	char[10]	Task name, aligned with configMAX_TASK_NAME_LEN=10
stackHighWaterMark	uint16	Minimum remaining stack space since task creation, in words
priority	uint8	Current task priority
state	uint8	Task state: 0=Running, 1=Ready, 2=Blocked, 3=Suspended, 4=Deleted
runtimeCounter	unsigned long long	Accumulated runtime, in microseconds, 64-bit no-overflow
cpuLoadPercent	uint8	CPU usage during the previous statistics window (0~100%)

System-level diagnostic fields (EcuM_DiagInfo_t):

Field Name	Type	Description
tasks[8]	EcuM_TaskDiag_t[8]	Diagnostic information for each active task
taskCount	uint8	Current number of active tasks
freeHeapSize	uint32	Currently available heap space, in bytes
minEverFreeHeapSize	uint32	Historical minimum available heap space, in bytes
totalHeapSize	uint32	Total heap size, currently configured as 16384 bytes
heapUsagePercent	uint8	Heap usage = (total - free) / total × 100%
totalRuntime	unsigned long long	Total system runtime, in microseconds, 64-bit
overallCpuLoad	uint8	Overall CPU load = 100% – IDLE task ratio
uptimeSeconds	uint32	System uptime, in seconds
updateCount	uint32	Diagnostic update counter
diagSkipCount	uint32	Number of skipped updates due to DWT overflow risk

5. Data Overflow Protection and Precision Drift Prevention Design

5.1 DWT 32-bit Overflow Avoidance Logic (The 10.73 s Trap Fix)

- Overflow point:** the 32-bit counter overflows about every 10.73 seconds at 400 MHz ($2^{32} / 400,000,000 \approx 10.73s$).
- Unsigned subtraction boundary property:** for uint32 subtraction new - old , as long as the interval between two samples is < 10.73 seconds, two's complement arithmetic in C guarantees a correct result even if one overflow occurs.
- Fail-safe protection** (EcuM_main_init.c:221-249): when the interval between two adjacent calls is ≥ 10 seconds, with 0.73 seconds reserved as safety margin, the current calculation is proactively abandoned, all baselines are reset, and

`diagSkipCount` is incremented. Under normal conditions, the diagnostic update period is 1 second, which is far smaller than the overflow period. This protection is intended only for abnormal cases, such as long blocking by a high-priority task.

5.2 Lossless 64-bit Cycle Accumulation

- The accumulated runtime of each task is stored as the **raw DWT cycle count** in `unsigned long long accumCycles` (64-bit).
- Conversion to microseconds through `/400ULL` is performed only when the final value is written into `g_ecuMDiag.tasks[i].runtimeCounter`.
- This avoids accumulated truncation error, or precision drift, caused by performing integer division on every delta.

6. System Task Map

The following is the complete list of runtime tasks in the current system, created through the initialization chain in `main()` :

Task Name	Priority	Stack Size (words)	Creation Location	Notes
PICC_Init_Task	4 (highest user task)	256	<code>picc_main.c:475</code>	One-time initialization task, self-deletes through <code>vTaskDelete(NULL)</code> after completion
App_Main_10ms	3	192	<code>picc_main.c:490</code>	PICC RX message handling, waiting on a blocking queue
ANIFC_CAN_10ms	2	256	<code>FlexCAN_Ip_main.c:325</code>	10 ms CAN periodic TX/RX + diagnostic update call entry
task_M7_0_10ms	2	256	<code>picc_main.c:501</code>	PICC stack/heartbeat/link + power state machine
CpuTest	1	128	<code>EcuM_CpuLoadTest.c:110</code>	Created only when <code>ECUM_CPULOAD_TEST_ENABLE=1</code>
IDLE	0 (lowest)	<code>configMINIMAL_STACK_SIZE=90</code>	Automatically created by the FreeRTOS kernel	Idle task, used as the reverse-deduction baseline for CPU load
Tmr_Svc	2	<code>configTIMER_TASK_STACK_DEPTH=180</code>	Automatically created by the FreeRTOS kernel	Software timer service task (<code>configUSE_TIMERS=1</code>)

Scheduler startup: `vTaskStartScheduler()` is called at `picc_main.c:523` , at the end of `PICC_Mian_Task()` .

main() initialization sequence (`EcuM_main_init.c:412`):

```
main()
├─ EcuM_Diag_Init()           ← Clear all diagnostic state
├─ EcuM_CpuLoadTest_Init()    ← Create the stress test task if enabled
├─ Mcu_Init()                 ← MCU driver initialization
├─ Platform_Init()            ← Platform initialization, including interrupt controller
├─ Port_Init() + Siul2_Port_Ip_Init() ← Pin initialization
├─ I2c_Init()                 ← I2C driver
├─ Pmic_driver_init()         ← PMIC power management chip
├─ SPI + TJA1145 initialization ← CAN transceiver initialization
├─ FlexCAN_Ip_Init/SetStartMode ← CAN controller initialization
├─ FlexCAN_Process_Init()      ← CAN MB configuration
├─ AINFC_CAN_Main_task()       ← Create the CAN 10 ms periodic task
└─ PICC_Mian_Task()           ← Create PICC tasks + vTaskStartScheduler()
```

7. CPU High-Load Stress Test Module

This module is independently encapsulated in `EcuM_CpuLoadTest.c` and is used to generate controllable artificial CPU load to verify the accuracy of the diagnostic system.

7.1 Build Switches

Macro Name	Default Value	Description
ECUM_CPULOAD_TEST_ENABLE	0U	0 = fully disabled (zero code / zero data footprint); 1 = enabled
ECUM_CPULOAD_TEST_PRIORITY	1U	Task priority, where 1 means just above IDLE

- The default value can be overridden on the compiler command line with `-DECUM_CPULOAD_TEST_ENABLE=1U`.

7.2 Working Mechanism

The `CpuLoadTest_Task` task, with stack size 128 words = 512 bytes, executes one scheduling cycle every 100 ms using `vTaskDelayUntil`:

1. Read `g_cpuLoadTestPercent` (volatile, in a non-cacheable section, directly modifiable from Trace32).
2. If it is > 0, convert the percentage into DWT cycles: `busyCycles = targetPercent × CYCLES_PER_MS`, where
`CYCLES_PER_MS = 400,000,000 / 1000 = 400,000`.
3. Busy-spin in a loop to consume the corresponding number of cycles:
`while ((DWT_CYCCNT_REG - startCycle) < busyCycles) { NOP; }`.
4. Suspend for the remaining time with `vTaskDelayUntil`.

7.3 Safety Clamp

- A built-in **95% upper clamp** is applied (`EcuM_CpuLoadTest.c:83`). Even if 100% is requested, the task will burn at most 95 ms per 100 ms period, keeping a 5% margin for critical system work such as watchdog servicing and interrupt response, preventing starvation-induced reset.

7.4 Trace32 Online Control

No code modification or firmware reflash is required. Directly operate from the Trace32 command line:

```
Var.Set g_cpuLoadTestPercent = 30    // Inject 30% CPU load
Var.Set g_cpuLoadTestPercent = 80    // Inject 80% CPU load
Var.Set g_cpuLoadTestPercent = 0     // Stop injection and return to idle
Var.View g_ecuMDiag                  // Observe diagnostic data to verify actual load
```

8. Public API Summary

Function Name	File	Description
vMainConfigureTimerForRunTimeStats()	EcuM_main_init.c:106	FreeRTOS callback: initialize the DWT cycle counter
ulMainGetRunTimeCounterValue()	EcuM_main_init.c:125	FreeRTOS callback: return the raw DWT cycle count value
EcuM_Diag_Init()	EcuM_main_init.c:171	Initialize the diagnostic subsystem, clear all trackers and baseline values. Must be called before vTaskStartScheduler() .
EcuM_Diag_Update()	EcuM_main_init.c:203	Periodically update diagnostic data, perform load-rate delta calculation, and refresh g_ecuMDiag . Must only be called from RTOS task context, never from ISR.
EcuM_CpuLoadTest_Init()	EcuM_CpuLoadTest.c:106	Create the CPU load test task. When ECUM_CPULOAD_TEST_ENABLE=0 , it compiles into an empty function.

9. Key Design Constraints and Notes

- Diagnostic array capacity:** at most ECUM_DIAG_MAX_TASKS = 8 tasks can be monitored. The current system has at most 7 tasks, including IDLE and Timer Service, which is already close to the limit. If new tasks are added, this macro and the size of s_taskStatusArray must be updated together.
- DWT safe interval:** the call interval of EcuM_Diag_Update() must be < 10 seconds. The current 1-second period is far below this limit.
- Non-cacheable section:** g_ecuMDiag , g_cpuLoadTestPercent , s_taskTracker , and s_taskStatusArray are all placed in the .mcal_bss_no_cacheable section to ensure Trace32 debug visibility.
- Task name length:** ECUM_DIAG_TASK_NAME_LEN = 10 , kept consistent with configMAX_TASK_NAME_LEN = 10 . Task names longer than 9 characters, including the trailing \0 , will be truncated.
- Impact of CPU load testing on the system:** the test task has priority 1, which is only above IDLE. Therefore, it only steals time from IDLE and does not affect scheduling of application tasks with priority ≥ 2. However, high load still reduces interrupt response margin.